Modern AI Systems Ontology

From Model Containers to Semantic Orchestration and Message-Passing Pipelines

Purpose. This document organizes modern AI applications and services into an ontology of layers. It starts with the abstract stack:

```
physical execution
    ↓
tensor runtimes
    ↓
model packaging ecosystems
    ↓
semantic orchestration ecosystems
    ↓
semantic transformation modules
    ↓
typed semantic message passing
    ↓
application orchestration ecosystems
    ↓
AI services / workflows / agents
```

The focus is on the middle layers where confusion often appears:

1. **Model packaging ecosystems** — `.safetensors`, GGUF, ONNX, TensorRT engines, etc.

2. **Semantic orchestration ecosystems** — Diffusers, Transformers, ComfyUI, Ollama, vLLM, LangChain, etc.

3. **Transformation modules** — VAE, UNet, DiT, CLIP, LoRA, scheduler, tokenizer, retriever, etc.

4. **Message / data types** — tokens, embeddings, latents, logits, KV cache, conditioning vectors, tool calls, retrieved chunks, etc.

---

0. The corrected ontology

LEVEL 0 — Physical execution substrate

This is the hardware-level compute substrate.

Examples:

- CPU scalar/SIMD execution
- NVIDIA CUDA GPUs
- AMD ROCm/HIP GPUs
- Apple Metal / Neural Engine
- Google TPU runtime
- specialized inference ASICs

Semantic role: none.

This layer only performs numerical computation.

```
numbers → numbers
```

LEVEL 1 — Tensor / neural runtimes

These are engines capable of executing tensor graphs or neural-network operations.

Examples:

- PyTorch
- TensorFlow
- JAX / XLA
- ONNX Runtime
- TensorRT
- TensorRT-LLM
- llama.cpp / GGML
- MLX
- OpenVINO
- TVM
- IREE
- MNN / NCNN / TFLite

Role:

```
tensor program → executed tensor program
```

These runtimes answer: **how is computation executed?**

They usually know about:

- tensors
- kernels
- memory layout
- quantization
- device placement
- batching
- graph optimization
- sometimes distributed execution

They usually do **not** know the full semantic meaning of the model pipeline. PyTorch knows `matmul`, `conv2d`, `attention`, and tensors. It does not inherently know that a tensor is “a latent image representation conditioned by a CLIP embedding at diffusion timestep t”.

LEVEL 2 — Model packaging ecosystems

These are serialization / container / deployment formats for models.

Examples:

- `.safetensors`
- PyTorch `.pt` / `.pth`
- TensorFlow SavedModel
- Keras `.keras` / HDF5 `.h5`
- ONNX `.onnx`
- GGUF
- TensorRT `.engine` / plan
- OpenVINO IR
- TFLite
- MLX weights
- Core ML `.mlmodel`
- NCNN / MNN / TVM artifacts
- model repository layouts such as Hugging Face model repos

Role:

```
model parameters + metadata + sometimes graph → loadable model artifact
```

They answer: **how is the model stored, exchanged, quantized, or deployed?**

LEVEL 3 — Semantic orchestration ecosystems

These are libraries, frameworks, UIs, or serving systems that understand the semantic structure of an AI task.

Examples:

- Hugging Face Diffusers
- Hugging Face Transformers
- ComfyUI
- AUTOMATIC1111
- InvokeAI
- Ollama
- vLLM
- SGLang

- Text Generation Inference
- LangChain
- LlamaIndex
- Haystack
- DSPy
- Semantic Kernel
- OpenAI Responses / Assistants style APIs
- Azure AI Foundry
- Amazon Bedrock
- Google Vertex AI / Model Garden

Role:

```
semantic task → coordinated model execution
```

They answer: **what components should interact, in what order, with what semantic messages?**

LEVEL 4 — Orchestration elements / transformation modules

These are the semantic building blocks inside pipelines.

Examples:

- tokenizer
- embedding layer
- transformer block
- attention layer
- MLP block
- language-model head
- sampler
- KV cache
- VAE encoder / decoder
- UNet
- diffusion transformer / DiT
- CLIP / T5 text encoder
- scheduler
- ControlNet
- IP-Adapter
- LoRA adapter
- reranker
- retriever

- vector database
- tool/function caller
- guardrail
- evaluator

Some of these are neural networks. Some are not.

Element	Neural network?	Main type
VAE	yes	learned transform
UNet / DiT	yes	learned denoising / flow transform
CLIP / T5 encoder	yes	learned semantic encoder
LoRA	not a standalone network	parameter delta / adapter
Scheduler	no	numerical controller
Tokenizer	no	symbolic preprocessor
Sampler	usually no	stochastic / deterministic controller
KV cache	no	runtime state
Retriever	often mixed	search / embedding subsystem
Guardrail	mixed	policy controller
Evaluator	mixed	quality / safety measurement

LEVEL 5 — Orchestration message / data types

These are the typed semantic objects passed between modules.

Examples:

- text strings
- tokens
- token IDs
- embeddings
- hidden states
- logits
- probabilities
- KV cache tensors
- latents
- noise tensors
- conditioning vectors
- attention masks
- attention maps

- image tensors
- audio tensors
- video tensors
- depth maps
- pose maps
- segmentation maps
- retrieved document chunks
- JSON tool calls
- function outputs
- agent state
- evaluation traces

This is one of the most important abstractions:

```
modern AI systems = typed semantic message-passing systems over tensors
```

LEVEL 6 — Application orchestration / AI platform ecosystems

This includes cloud studios, foundries, and production AI application frameworks.

Examples:

- OpenAI Platform / ChatGPT / Responses API
- Azure AI Foundry
- Anthropic Console / Claude API
- Google Vertex AI / Gemini Enterprise Agent Platform
- Amazon Bedrock
- Databricks Mosaic AI
- Snowflake Cortex
- IBM watsonx
- Hugging Face Inference Endpoints
- Replicate
- Modal
- Together AI
- GroqCloud
- Cerebras Inference

Role:

```
business / user task → AI application workflow → monitored service
```

These platforms usually orchestrate:

- model endpoints
- prompts
- agents
- tools
- vector stores
- RAG pipelines
- evaluation
- monitoring
- deployment
- billing
- access control
- safety and governance

They usually expose **service objects**, not raw tensors.

---

1. Model packaging ecosystems

A model package is not necessarily “the model” in the full semantic sense. It is usually a storage or deployment representation of weights, metadata, and sometimes computation graphs.

A useful distinction:

```
weights-only container
vs
graph + weights container
vs
compiled inference artifact
vs
repository/package layout
```

1.1 `.safetensors`

Layer: model packaging ecosystem

Common domain: PyTorch / Hugging Face / diffusion / LLM training checkpoints

Typical users: ComfyUI, Diffusers, Transformers, PEFT, training scripts

`safetensors` is a safe and fast tensor serialization format, designed to avoid arbitrary code execution risks associated with pickle-based formats and to support fast loading / zero-copy behavior where possible. Hugging Face describes it as a simple, safe tensor storage format. [1]

What it contains

- named tensors

- tensor shapes
- tensor dtypes
- raw tensor data
- optional metadata

It does **not** by itself contain a full execution graph.

Semantic I/O

At the file level:

```
parameter_name → tensor
```

Example:

```
"model.diffusion_model.input_blocks.0.0.weight" → FP16 tensor
```

At the runtime level, a framework reconstructs a model architecture and loads matching tensors into it.

```
architecture class + safetensors state_dict → runnable neural module
```

Where it appears

- Stable Diffusion checkpoints
- SDXL
- FLUX components
- LoRA adapters
- VAE files
- CLIP / T5 text encoders
- LLM checkpoints on Hugging Face
- sharded model weights

Strengths

- safe compared with pickle checkpoints
- simple
- fast loading
- widely used in Hugging Face ecosystems
- good for training and fine-tuning workflows
- flexible for arbitrary PyTorch modules

Weaknesses

- usually not self-sufficient
- architecture code is external
- tokenizer is external
- scheduler / sampler logic is external
- not automatically optimized for inference
- quantization metadata support depends on ecosystem conventions

---

1.2 PyTorch `.pt` / `.pth` / `.bin`

Layer: model packaging ecosystem
Common domain: research, training, older Hugging Face models

These are PyTorch serialization files. Historically, many models were distributed as `pytorch_model.bin`.

What they contain

Depending on saving method:

- state dictionary of tensors
- full Python object graph
- optimizer states
- training metadata
- arbitrary pickled Python structures

Semantic I/O

```
serialized Python/PyTorch object → loaded Python/PyTorch object
```

or:

```
state_dict parameter names → tensors
```

Strengths

- very flexible
- native to PyTorch
- can store training state

Weaknesses

- pickle-based loading can be unsafe when files are untrusted
- less deployment-friendly
- not portable outside PyTorch
- less explicit than graph formats like ONNX

1.3 GGUF

Layer: model packaging ecosystem

Common domain: llama.cpp, Ollama, local LLM inference

GGUF is a binary format for storing models for inference with GGML and GGML-based executors. The GGML documentation describes it as designed for fast loading/saving and ease of reading, with models usually trained in PyTorch and converted to GGUF for inference. [2] Hugging Face also describes GGUF as optimized for quick model loading and inference with GGML-family executors. [3]

What it contains

Usually:

- quantized tensors
- architecture metadata
- tokenizer metadata
- normalization constants
- RoPE / context-length parameters
- vocabulary
- special tokens
- quantization type metadata
- sometimes chat-template metadata

Semantic I/O

For LLMs:

```
prompt text
    ↓ tokenizer
token IDs
    ↓ transformer
logits
    ↓ sampler
next token
```

At file level:

```
GGUF file → self-describing local inference artifact
```

Strengths

- very deployment-friendly
- excellent for CPU and mixed CPU/GPU inference
- strong quantization ecosystem
- often relatively self-contained
- ideal for local LLM serving through llama.cpp / Ollama

Weaknesses

- mainly inference-oriented
- not ideal for training
- originally LLM-centric, though use is expanding
- conversion may lose some framework-specific flexibility
- not the native format for most diffusion training workflows

---

1.4 ONNX

Layer: graph + weights model format

Common domain: cross-framework inference and deployment

ONNX represents computation as a graph of typed operators, with inputs, outputs, nodes, initializers, and attributes. ONNX documentation describes models in terms of operators such as `MatMul` and `Add`, with strongly typed shapes and graph structure. [4]

What it contains

- computation graph
- nodes / operators
- inputs and outputs
- tensor initializers, often weights
- attributes
- opset version
- type and shape information

Semantic I/O

```
typed input tensors → ONNX graph → typed output tensors
```

Example:

```
Input: float[M,K] x
Weights: float[K,N] A
Bias: float[N] b
Output: y = Add(MatMul(x, A), b)
```

Strengths

- portable
- graph-based
- can be optimized by ONNX Runtime, TensorRT, OpenVINO, etc.
- useful for production deployment
- explicit typed I/O

Weaknesses

- export can be difficult for dynamic models
- not every PyTorch operation maps cleanly
- LLM generation loops often need extra runtime logic
- large multimodal / agentic systems are not captured as one simple graph

---

1.5 TensorRT engine / plan

Layer: compiled inference artifact

Common domain: NVIDIA production inference

TensorRT is NVIDIA’s SDK for optimizing and accelerating deep learning inference on NVIDIA GPUs, supporting optimized deployment from frameworks such as PyTorch, TensorFlow, and ONNX with mixed precision and dynamic shape support. [5]

What it contains

- optimized graph
- fused kernels
- memory plan
- tactic choices
- precision choices
- GPU-specific execution plan

Semantic I/O

```
input tensors → compiled optimized engine → output tensors
```

Strengths

- very fast on NVIDIA GPUs
- excellent kernel fusion
- optimized memory planning
- FP16 / BF16 / FP8 / INT8 paths
- strong production inference performance

Weaknesses

- hardware/vendor specific
- often version-specific
- not easy to edit
- not training-oriented
- less transparent than PyTorch / ONNX

1.6 TensorRT-LLM artifacts

Layer: LLM-specific optimized inference ecosystem

Common domain: high-throughput NVIDIA LLM serving

TensorRT-LLM adds LLM-specific optimizations such as custom attention/GEMM kernels, speculative decoding, parallelism strategies, and runtime optimizations for efficient inference on NVIDIA GPUs. [6]

What it contains / manages

- optimized LLM graph artifacts
- custom kernels
- KV-cache management
- tensor/pipeline/expert parallelism
- quantization
- serving runtime components

Semantic I/O

```
tokens + KV cache → optimized transformer decode → logits / next token
```

Strengths

- high performance on NVIDIA
- production-grade scaling
- LLM-specific optimization

Weaknesses

- NVIDIA-centric
- more complex deployment
- less flexible for experimentation than PyTorch

1.7 TensorFlow SavedModel

Layer: graph + weights package
Common domain: TensorFlow production deployment

What it contains

- computation graph
- variables / weights
- signatures
- assets
- serving functions

Semantic I/O

```
named input signature → TensorFlow graph → named output signature
```

Strengths

- mature production deployment
- TensorFlow Serving support
- explicit signatures

Weaknesses

- less dominant in frontier LLM/diffusion research than PyTorch

- conversion can be involved
- Python research ecosystem has shifted strongly toward PyTorch

---

1.8 Keras `.keras` / HDF5 `.h5`

Layer: model packaging
Common domain: Keras / TensorFlow workflows

What it contains

- model architecture config
- weights
- optimizer state, optionally
- training configuration

Semantic I/O

```
Keras model package → loaded Keras model → tensors in/out
```

Strengths

- user-friendly
- compact for Keras models
- good for classical deep learning workflows

Weaknesses

- not central for modern LLM/diffusion ecosystems
- less flexible for custom research code

1.9 OpenVINO IR

Layer: optimized deployment representation
Common domain: Intel CPU/GPU/NPU inference

What it contains

- graph XML / binary weights, depending on version
- optimized network representation
- precision and layout information

Semantic I/O

```
input tensors → OpenVINO execution graph → output tensors
```

Strengths

- strong CPU inference
- Intel hardware optimization
- useful for edge and desktop deployment

Weaknesses

- vendor ecosystem
- not a training format
- conversion required

1.10 TFLite

Layer: mobile/edge inference format
Common domain: Android, mobile, embedded

What it contains

- flatbuffer model graph
- weights
- quantization metadata
- operator list

Semantic I/O

```
small typed tensors → mobile inference graph → small typed tensors
```

Strengths

- compact
- mobile-friendly
- quantization support

Weaknesses

- limited operator coverage
- not ideal for large generative AI systems
- conversion path can be restrictive

1.11 Core ML `.mlmodel` / `.mlpackage`

Layer: Apple deployment format
Common domain: iOS/macOS deployment

What it contains

- model graph
- weights
- metadata
- input/output signatures

Semantic I/O

```
Apple app input tensor/image/text → Core ML model → output
```

Strengths

- integrated into Apple platforms
- uses Apple Neural Engine / Metal acceleration
- production-friendly for apps

Weaknesses

- Apple-platform specific
- conversion required
- less flexible than PyTorch for research

1.12 MLX weights

Layer: Apple Silicon inference/training ecosystem
Common domain: local LLMs and diffusion on Apple Silicon

What it contains

- arrays / tensors in MLX-compatible layouts
- model configs
- sometimes converted Hugging Face weights

Semantic I/O

```
MLX model + weights → Apple Silicon execution
```

Strengths

- Apple Silicon optimized
- Pythonic
- increasingly popular for local inference

Weaknesses

- Apple-specific
- younger ecosystem
- not as universal as PyTorch / ONNX

1.13 NCNN / MNN / TVM / IREE artifacts

Layer: optimized inference / compiler artifacts
Common domain: mobile, embedded, custom deployment

What they contain

- optimized graph
- weights

- operator schedule / compiled kernels
- target-specific metadata

Semantic I/O

```
input tensors → optimized compiled graph → output tensors
```

Strengths

- efficient deployment
- can target constrained devices
- useful for edge AI

Weaknesses

- conversion complexity
- harder debugging
- less flexible than high-level frameworks

---

2. Semantic orchestration ecosystems

A semantic orchestration ecosystem is not merely a file format or a tensor runtime. It knows **how components should be connected** for a particular AI task.

2.1 Hugging Face Diffusers

Layer: semantic orchestration ecosystem

Domain: diffusion / flow-based image, video, audio generation

Diffusers provides diffusion pipelines, interchangeable schedulers, and pretrained model building blocks. Hugging Face describes a `DiffusionPipeline` as wrapping components such as UNets or diffusion transformers, text encoders, VAEs, and schedulers into a usable API while preserving flexibility. [7] The project also explicitly presents its core components as pipelines, interchangeable noise schedulers, and pretrained models. [8]

What it orchestrates

- text encoders
- UNet or DiT denoisers

- VAE encoder/decoder
- schedulers
- samplers
- ControlNet
- LoRA
- IP-Adapter
- image processors
- safety checkers
- video frame processors

Semantic I/O

For text-to-image latent diffusion:

```
text prompt
    ↓ tokenizer + text encoder
text embeddings
    ↓ conditioning
initial noise latent
    ↓ denoiser + scheduler loop
final latent
    ↓ VAE decoder
image
```

Mathematical core

Diffusion / flow systems learn a transformation between noise and data.

A simplified diffusion denoising view:

```
x_t = noisy sample at timestep t
εθ(x_t, t, c) = neural prediction of noise / velocity / score conditioned by c
scheduler step: x_t → x_{t-1}
```

For rectified flow / flow matching, the model often learns a velocity field:

```
vθ(x_t, t, c) ≈ dx/dt
```

The scheduler / sampler numerically integrates this learned vector field.

Why Diffusers is a separate layer

PyTorch executes tensor operations. Diffusers knows that:

- this tensor is a latent
- this module is a VAE
- this loop is denoising
- this scheduler determines the timestep trajectory
- this text encoder produces conditioning

So Diffusers belongs to the **semantic orchestration layer**, not the tensor runtime layer.

2.2 Hugging Face Transformers

Layer: semantic orchestration ecosystem

Domain: transformers for text, vision, audio, multimodal models

Transformers acts as a model-definition framework for state-of-the-art models in text, computer vision, audio, video, and multimodal domains. [9] Its pipeline API abstracts preprocessing, model execution, and postprocessing for tasks like text generation, question answering, classification, ASR, and document QA. [10]

What it orchestrates

- tokenizer
- model architecture
- generation loop
- logits processors
- stopping criteria
- attention masks
- KV cache
- task-specific heads
- postprocessing

Semantic I/O

For LLM text generation:

```
text
    ↓ tokenizer
token IDs
    ↓ transformer
hidden states
    ↓ LM head
logits
    ↓ sampler
next token
    ↓ detokenizer
text
```

Mathematical core

Autoregressive LLMs estimate:

```
P(x_1, ..., x_n) = Π_i P(x_i | x_<i)
```

At inference:

```
hidden = Transformer(tokens, KV cache)
logits = W_out hidden
next_token ~ Sampling(softmax(logits / temperature))
```

Difference from Diffusers

Transformers usually orchestrates a more monolithic model:

```
tokenizer → transformer → logits → sampler
```

Diffusers orchestrates a more heterogeneous pipeline:

```
text encoder + latent + denoiser + scheduler + VAE
```

2.3 ComfyUI

Layer: visual semantic orchestration / graph execution

Domain: diffusion, image/video workflows, increasingly multimodal workflows

ComfyUI is best understood as:

```
visual typed semantic tensor graph programming
```

It exposes components as nodes and message types as graph edges.

What it orchestrates

- checkpoint loaders
- CLIP encoders
- VAEs
- UNet / DiT models
- LoRAs
- ControlNets
- samplers
- schedulers
- latent image nodes
- image processing nodes
- video nodes
- upscalers
- external plugins

Semantic I/O

```
node output socket → typed semantic object → node input socket
```

Examples:

```
CLIP text encode → CONDITIONING
Empty latent image → LATENT
KSampler → LATENT
VAE decode → IMAGE
Load LoRA → modified MODEL + modified CLIP
```

Mathematical role

ComfyUI itself does not define the neural math. It defines graph execution:

```
DAG of semantic operations → scheduled execution → cached outputs
```

It is an orchestration UI/runtime over PyTorch/diffusion components.

2.4 AUTOMATIC1111 Stable Diffusion WebUI

Layer: diffusion application UI

Domain: Stable Diffusion-style image generation

What it orchestrates

- checkpoint
- VAE
- prompt / negative prompt
- sampler
- scheduler
- LoRA
- embeddings
- ControlNet extensions
- img2img / inpainting pipelines

Difference from ComfyUI

AUTOMATIC1111 is mostly parameter-panel driven:

```
fixed pipeline + configurable parameters
```

ComfyUI is graph-driven:

```
user-defined pipeline DAG
```

2.5 InvokeAI

Layer: diffusion application / workflow environment
Domain: image generation and creative workflows

What it orchestrates

- diffusion models
- prompts
- canvas workflows
- image-to-image
- inpainting
- model management
- some workflow graph capabilities

Role

InvokeAI is more artist/product oriented than ComfyUI, which is more graph-programming oriented.

2.6 Ollama

Layer: local LLM model manager + serving orchestration
Domain: GGUF-based local LLM serving

Ollama is not just a runtime. It is a higher-level local serving ecosystem around llama.cpp-style inference.

What it orchestrates

- model download
- GGUF model storage
- Modelfile configuration
- prompt templates
- chat templates
- local HTTP API
- inference process
- model loading/unloading

Semantic I/O

```
chat message
    ↓ prompt template / tokenizer
tokens
    ↓ llama.cpp runtime
logits
    ↓ sampler
assistant message
```

Why it feels “packaged”

Ollama hides most pipeline components. Compared with diffusion:

```
LLM: tokenizer + transformer + sampler
```

is relatively compact and monolithic.

2.7 llama.cpp / GGML

Layer: tensor runtime + LLM inference runtime

Domain: local quantized inference

llama.cpp requires models to be in GGUF format and can convert models from other formats into GGUF. [11]

What it orchestrates

- tokenization
- transformer forward pass
- KV cache
- quantized matrix multiplication
- sampling
- CPU/GPU offload

Semantic I/O

```
tokens + KV cache → logits + updated KV cache
```

Mathematical core

Autoregressive transformer inference with efficient quantized linear algebra.

2.8 vLLM

Layer: high-throughput LLM serving engine

Domain: production LLM serving

vLLM emphasizes high-throughput serving with PagedAttention, continuous batching, chunked prefill, prefix caching, CUDA/HIP graphs, and broad quantization support. [12]

What it orchestrates

- model loading
- request batching
- KV cache memory management
- prefill/decode scheduling
- prefix cache
- quantization
- distributed execution
- OpenAI-compatible APIs

Semantic I/O

```
many concurrent chat/completion requests
    ↓ scheduler
batched token execution
    ↓ transformer
streamed completions
```

Mathematical / systems core

The neural math is still transformer inference. The key innovation is systems-level scheduling:

```
KV cache treated like paged memory
continuous batching keeps GPU busy
```

2.9 SGLang

Layer: structured LLM program serving

Domain: high-performance structured generation, agents, multimodal LLMs

SGLang is a high-performance serving framework for large language and multimodal models, designed for low latency and high throughput; its features include RadixAttention, prefix caching, prefill-decode disaggregation, speculative decoding, continuous batching, and structured outputs. [13]

What it orchestrates

- structured generation programs
- multi-call LLM workflows
- prefix sharing
- KV cache reuse
- tool-like control flow
- JSON / constrained output
- distributed serving

Semantic I/O

```
structured LLM program
    ↓ runtime scheduler
multiple coordinated generation calls
    ↓
structured outputs / text / tool calls
```

Role

SGLang sits between:

```
LLM serving runtime
```

and:

```
programming language for LLM workflows
```

2.10 Text Generation Inference (TGI)

Layer: production LLM serving
Domain: Hugging Face model serving

What it orchestrates

- model loading
- tokenizer
- batching
- tensor parallelism
- streaming output
- inference API
- safetensors/sharded weights

Semantic I/O

```
HTTP request → tokens → model → generated tokens → HTTP stream
```

2.11 LangChain

Layer: LLM application orchestration

Domain: agents, tools, RAG, chains

What it orchestrates

- prompt templates
- LLM calls
- tools
- memory
- retrievers
- vector stores
- agents
- output parsers

Semantic I/O

```
user query
    ↓ prompt / chain / agent
LLM calls + tool calls + retrieval
    ↓
answer / action / structured output
```

Not a tensor runtime

LangChain usually does not run neural networks itself. It orchestrates services and model APIs.

2.12 LlamaIndex

Layer: data / RAG orchestration

Domain: connecting LLMs to private data

What it orchestrates

- document loaders
- chunkers
- embeddings
- vector indexes
- retrievers
- rerankers
- query engines
- response synthesizers

Semantic I/O

```
documents → chunks → embeddings → index
query → query embedding → retrieved chunks → LLM answer
```

2.13 Haystack

Layer: RAG / NLP pipeline orchestration

Domain: enterprise search, RAG, QA

What it orchestrates

- retrievers
- readers
- generators
- document stores
- rankers
- pipelines

Semantic I/O

```
query + document store → retrieved docs → generated answer
```

2.14 DSPy

Layer: programming / optimization framework for LLM pipelines

Domain: optimizing prompts and modular LLM programs

What it orchestrates

- declarative modules
- LLM calls
- teleprompters / optimizers
- examples / metrics
- compiled prompting strategies

Semantic I/O

```
task specification + examples + metric → optimized LLM program
```

2.15 Semantic Kernel

Layer: application orchestration / agent framework

Domain: enterprise LLM integration

What it orchestrates

- skills/plugins
- prompts
- planners
- tools
- memory
- function calling
- agents

Semantic I/O

```
business task → planner/tool calls/LLM calls → result
```

2.16 OpenAI Platform / Responses API

Layer: hosted AI application orchestration

Domain: hosted frontier model access, tools, agents, multimodal APIs

OpenAI's current tool documentation describes built-in tools, function calling, tool search, remote MCP servers, web search, and file search. [14] File search specifically provides retrieval over uploaded files through semantic and keyword search using vector stores. [15]

What it orchestrates

- hosted model endpoint
- prompt / instructions
- tools
- file search / vector stores
- web search
- code execution
- function calling
- multimodal inputs/outputs
- safety/policy infrastructure

Semantic I/O

```
developer/user messages + tools + files
    ↓ hosted model orchestration
model response + tool calls + retrieved context
```

File formats

Externally, OpenAI APIs typically expose service-level objects:

- messages
- files
- vector stores
- tool definitions
- JSON schemas
- images/audio/text

They generally do **not** expose raw model containers such as GGUF or safetensors to users.

2.17 Azure AI Foundry / Microsoft Foundry

Layer: cloud AI application orchestration / enterprise AI platform

Domain: model catalog, agents, tools, evaluation, deployment, monitoring

Microsoft Foundry documentation describes Foundry Agent Service as a managed platform for building, deploying, and scaling AI agents using frameworks such as Agent Framework, LangGraph, or user code, with many models from the Foundry model catalog. [16] Microsoft Foundry also includes observability, evaluation, monitoring, and agentic workflow evaluation. [17]

What it orchestrates

- model catalog
- model deployments
- prompt agents
- hosted agents
- tools
- business processes
- RAG / knowledge grounding
- evaluation
- monitoring
- content safety
- tracing
- access control
- enterprise deployment

Semantic I/O

```
enterprise task / app request
    ↓ agent + tools + knowledge + model endpoint
business action / answer / workflow result
```

File/model ecosystem relation

At the user level:

```
model endpoint, dataset, tool, agent, evaluator
```

not:

```
raw tensor, VAE tensor, UNet weight
```

But lower-level Azure ML workflows can host custom containers and models in formats such as PyTorch, ONNX, MLflow, etc.

2.18 Amazon Bedrock

Layer: hosted foundation-model platform

Domain: model access, agents, knowledge bases, custom model import

Amazon Bedrock Knowledge Bases are designed to give foundation models and agents contextual information from private data sources. [18] Bedrock also supports custom model import for customized foundation models created elsewhere. [19]

What it orchestrates

- foundation model endpoints
- agents
- action groups
- knowledge bases
- retrieval
- data sources
- guardrails
- custom model import
- inference APIs

Semantic I/O

```
user request
    ↓ Bedrock agent / model / knowledge base
retrieved context + model output + tool/action result
```

2.19 Google Vertex AI / Model Garden

Layer: cloud AI / ML platform

Domain: model discovery, tuning, deployment, custom training, hosted Gemini and open models

Google Vertex AI documentation describes Model Garden as a way to discover, test, customize, and deploy Google proprietary and select open-source generative AI models and LLMs. [20]

What it orchestrates

- Model Garden
- Gemini models
- open model deployments
- custom training
- endpoints

- pipelines
- evaluation
- feature stores / data integrations
- agent platform components

Semantic I/O

```
model selection + data + deployment config
    ↓ Vertex AI platform
model endpoint / tuned model / application workflow
```

---

3. Transformation modules / orchestration elements

This section describes the important semantic modules used inside orchestration systems.

3.1 Tokenizer

Type: algorithmic symbolic encoder

Neural network? no

Common ecosystems: Transformers, Ollama, vLLM, OpenAI APIs, llama.cpp

Purpose

Convert text into discrete token IDs that a language model can process.

I/O

```
string → token IDs
token IDs → string
```

Example:

```
"Hello world" → [15496, 995]
```

Mathematical view

A tokenizer is usually a learned or constructed discrete segmentation map:

```
T: text → ■^n
```

It is not neural inference, but it defines the symbolic alphabet over which the LLM operates.

Parameters / artifacts

- vocabulary
- merge rules, e.g. BPE
- special tokens
- chat template
- normalization rules

3.2 Text embedding model

Type: learned semantic encoder
Neural network? yes
Common ecosystems: RAG, search, recommendation, clustering

Purpose

Map text into a continuous semantic vector space.

I/O

```
text / tokens → embedding vector
```

Example:

```
"phase transition in tin" → ■^d vector
```

Mathematical view

```
e = fθ(tokens)
```

Similarity is often computed by cosine similarity:

```
sim(a,b) = (a · b) / (||a|| ||b||)
```

Used for

- vector search
- clustering
- RAG retrieval
- semantic deduplication

- reranking pipelines

3.3 Transformer block

Type: learned sequence transformation module

Neural network? yes

Common ecosystems: LLMs, vision transformers, multimodal models, DiT

Purpose

Transform contextual token representations.

I/O

```
hidden states + attention mask + KV cache → updated hidden states + updated KV cache
```

Mathematical view

Simplified:

```
Attention(Q,K,V) = softmax(QK■ / √d) V
```

Then:

```
x' = x + Attention(LN(x))
x'' = x' + MLP(LN(x'))
```

Parameters

- attention projection matrices
- MLP matrices
- layer norm parameters
- positional encoding / RoPE parameters

3.4 Attention module

Type: learned routing / mixing operation

Neural network? partly; uses learned projections

Common ecosystems: LLMs, CLIP, DiT, multimodal models

Purpose

Let each token or patch attend to other tokens or patches.

I/O

```
queries, keys, values → context vectors
```

Mathematical core

```
Q = XW_Q
K = XW_K
V = XW_V
Attention = softmax(QK■ / √d_k)V
```

Semantic role

Attention is a differentiable message routing mechanism.

3.5 KV cache

Type: runtime state

Neural network? no

Common ecosystems: LLM serving

Purpose

Store previous keys and values during autoregressive generation so the model does not recompute all previous tokens at every step.

I/O

```
past K,V + new token hidden state → updated K,V + logits
```

Systems role

KV cache is often the main memory bottleneck in LLM serving. vLLM’s PagedAttention is specifically about efficient KV cache memory management. [12]

3.6 LM head

Type: learned output projection

Neural network? yes

Common ecosystems: LLMs

Purpose

Convert final hidden state into vocabulary logits.

I/O

```
hidden state → logits over vocabulary
```

Mathematical view

```
logits = h W_vocab■ + b
```

Then:

```
probabilities = softmax(logits)
```

3.7 Sampler / decoding controller

Type: algorithmic controller

Neural network? no

Common ecosystems: LLMs, diffusion systems

Purpose in LLMs

Choose the next token from logits.

I/O

```
logits → next token
```

Methods

- greedy decoding
- temperature sampling
- top-k
- top-p / nucleus sampling
- beam search
- repetition penalty
- grammar-constrained sampling
- JSON-schema constrained sampling

Mathematical view

```
p_i = softmax(logits_i / T)
next_token ~ filtered categorical distribution
```

3.8 VAE — Variational Autoencoder

Type: learned latent-space encoder/decoder

Neural network? yes

Common ecosystems: Stable Diffusion, SDXL, FLUX-like latent image systems

Purpose

A VAE compresses images into a smaller latent representation and decodes latents back into images. In latent diffusion, the denoising model operates in latent space rather than pixel space to reduce computation.

I/O

Encoder:

```
image tensor → latent tensor
```

Decoder:

```
latent tensor → image tensor
```

In ComfyUI terms:

```
IMAGE → VAE Encode → LATENT
LATENT → VAE Decode → IMAGE
```

Mathematical view

A variational autoencoder learns:

```
qϕ(z|x)    encoder distribution
pθ(x|z)    decoder distribution
```

Training optimizes a reconstruction term plus a KL regularization term:

```
L = E_q[log pθ(x|z)] - KL(qϕ(z|x) || p(z))
```

In practical latent diffusion, the VAE creates a compressed spatial latent such as:

```
image: 1024 × 1024 × 3
latent: 128 × 128 × C
```

depending on compression factor and channels.

What parameters it contains

- convolutional encoder weights
- convolutional decoder weights
- normalization parameters
- latent scaling factor
- sometimes quantization/post-processing config

Why it matters

The VAE determines:

- compression quality
- reconstruction sharpness
- color behavior
- detail preservation
- latent geometry
- VRAM/computation requirements

3.9 Latent tensor

Type: message/data type

Neural network? no

Common ecosystems: diffusion

Purpose

Represent an image or video in compressed continuous feature space.

I/O role

```
noise latent → denoising steps → clean latent → VAE decode → image
```

Mathematical view

Latents live in a learned manifold:

```
z ∈ ■^(H/f × W/f × C)
```

where `f` is the spatial compression factor.

3.10 UNet denoiser

Type: learned denoising model

Neural network? yes

Common ecosystems: Stable Diffusion 1.x/2.x, SDXL

Purpose

Predict how to denoise a noisy latent at timestep `t`, conditioned on text/image guidance.

I/O

```
noisy latent z_t + timestep t + conditioning c → predicted noise / velocity / denoised latent
```

Mathematical view

Common prediction target:

```
εθ(z_t, t, c) ≈ noise ε
```

or velocity:

```
vθ(z_t, t, c)
```

Scheduler uses this prediction to compute the next latent:

```
z_t → z_{t-1}
```

What parameters it contains

- convolutional downsampling blocks

- bottleneck blocks
- upsampling blocks
- residual blocks
- self-attention / cross-attention layers
- text-conditioning projections
- normalization layers

Why UNet shape

UNet combines:

- low-level spatial detail from early layers
- high-level semantic context from deeper layers
- skip connections preserving spatial structure

3.11 DiT — Diffusion Transformer

Type: learned denoising / flow model
Neural network? yes
Common ecosystems: Stable Diffusion 3, FLUX, modern image/video generation

Purpose

Replace UNet-style convolutional denoisers with transformer-based architectures operating on latent patches.

I/O

```
latent patches + timestep + conditioning → denoising / velocity prediction
```

Mathematical view

The latent is patchified:

```
z ∈ ■^(H×W×C) → sequence of latent patches
```

Then transformer blocks process the sequence using attention.

Why it matters

Transformers scale well with:

- model size

- multimodal conditioning

- long-range spatial dependencies

- unified text/image token handling

3.12 FLUX

Type: model family / architecture ecosystem

Neural network? yes, but “FLUX” names a full model family, not a single module

Common ecosystem: Black Forest Labs FLUX.1 models, ComfyUI, Diffusers

FLUX.1 is commonly described as a rectified-flow transformer trained in the latent space of an image encoder. [21] Related work on high-resolution rectified flow transformers describes flow-matching approaches for text-to-image synthesis. [22]

What FLUX is conceptually

FLUX is not a file format and not just a scheduler. It is a **text-to-image generative model family** based around rectified flow / flow matching and transformer-based diffusion-like architecture.

I/O

```
text conditioning + initial latent/noise + timestep
    → flow/velocity prediction
    → progressively transformed latent
    → decoded image
```

Mathematical view

Instead of classic diffusion’s discrete denoising formulation, rectified flow learns a vector field that transports noise toward data:

```
dx/dt = vθ(x_t, t, c)
```

Sampling numerically integrates:

```
x_0 noise → x_1 data-like latent
```

or the reverse depending on convention.

What parameters it contains

Depending on implementation:

- transformer blocks

- multimodal attention layers
- text-conditioning projections
- timestep embeddings
- feed-forward layers
- normalization parameters
- possibly separate text encoders and VAE-like image encoder/decoder components in the full pipeline

What FLUX works with

- text encoders such as CLIP/T5 depending on variant
- latent image representations
- flow-matching scheduler/sampler
- VAE/autoencoder decoder
- LoRA adapters
- sometimes quantized or split model formats in ComfyUI

3.13 Scheduler

Type: numerical controller

Neural network? no

Common ecosystems: Diffusers, ComfyUI, diffusion UIs

Diffusers documentation describes schedulers as functions for the diffusion process that take the model output and timestep to return a denoised sample; timesteps determine where in the diffusion process the step occurs. [23]

Purpose

Control the trajectory through noise/latent space.

I/O

```
current latent z_t
+ model prediction
+ timestep t
+ scheduler parameters
→ next latent z_{t-1}
```

Mathematical view

A scheduler is a numerical integrator for a learned denoising or flow equation.

Simplified diffusion step:

```
z_{t-1} = SchedulerStep(z_t, εθ(z_t,t,c), t)
```

For flow matching:

```
z_{t+Δt} = z_t + Δt · vθ(z_t,t,c)
```

Common scheduler/sampler families

- DDPM
- DDIM
- Euler
- Euler ancestral
- LMS
- Heun
- DPM-Solver
- DPM++ 2M
- UniPC
- DEIS
- PNDM
- LCMScheduler
- FlowMatchEulerDiscreteScheduler
- EDM schedulers

Parameters

- timestep schedule
- sigma schedule
- number of steps
- prediction type: epsilon / sample / velocity
- stochasticity parameter
- solver order
- ancestral noise amount
- guidance scale interaction

Why it matters

The same model with different schedulers can change:

- speed
- sharpness

- stability
- creativity
- prompt adherence
- artifact behavior

3.14 CLIP text encoder

Type: learned multimodal semantic encoder
Neural network? yes
Common ecosystems: Stable Diffusion, SDXL, image-text retrieval

Purpose

Encode text into semantic conditioning vectors aligned with image concepts.

I/O

```
prompt tokens → text embeddings / conditioning
```

Mathematical role

CLIP-style training aligns image and text embeddings in a shared space using contrastive learning.

In diffusion:

```
conditioning c = CLIP_text_encoder(prompt)
```

Then cross-attention injects `c` into UNet/DiT.

Parameters

- token embedding matrix
- transformer encoder layers
- positional embeddings
- projection layers
- layer norms

3.15 T5 text encoder

Type: learned text encoder

Neural network? yes

Common ecosystems: Imagen-like systems, SD3, FLUX variants

Purpose

Provide rich text conditioning, often better at long and structured prompts than older CLIP-only setups.

I/O

```
prompt tokens → sequence embeddings
```

Role in diffusion

The image generator conditions on text encoder hidden states.

3.16 Classifier-Free Guidance (CFG)

Type: algorithmic guidance method

Neural network? no, but uses neural model outputs

Common ecosystems: diffusion

Purpose

Strengthen conditional generation by comparing conditional and unconditional model predictions.

I/O

```
conditional prediction + unconditional prediction + guidance scale
→ guided prediction
```

Mathematical view

```
ε_guided = ε_uncond + s · (ε_cond - ε_uncond)
```

where `s` is guidance scale.

Parameters

- guidance scale
- negative prompt / unconditional conditioning
- sometimes rescale factor

3.17 Negative prompt

Type: conditioning input

Neural network? no

Common ecosystems: diffusion UIs

Purpose

Provide an unconditional or anti-conditioning direction used by CFG.

I/O

```
negative text → negative conditioning vector
```

Conceptual role

It does not “forbid” concepts logically. It changes the vector direction used during guided denoising.

3.18 LoRA

Type: low-rank parameter adapter

Neural network? not standalone; trainable parameter delta inserted into neural layers

Common ecosystems: LLM fine-tuning, diffusion style/character adapters

LoRA freezes the base model weights and injects trainable low-rank decomposition matrices into layers, greatly reducing trainable parameters. [24]

Purpose

Adapt a large model cheaply without storing a full fine-tuned copy.

I/O

LoRA modifies a layer transformation.

Original linear layer:

```
y = W x
```

LoRA-augmented layer:

```
y = (W + ΔW)x
ΔW = B A
```

where:

```
A ∈ ■^(r×d)
B ∈ ■^(k×r)
r << min(d,k)
```

What it contains

- small low-rank matrices
- scaling factor alpha
- target layer names
- sometimes metadata: trigger words, rank, base model

What it works with

- UNet attention layers
- DiT layers
- CLIP text encoder layers
- LLM attention/MLP layers

Important classification

LoRA is **not** a message type.

It is a **parameter delta adapter**.

3.19 ControlNet

Type: learned conditional control module

Neural network? yes

Common ecosystems: diffusion

ControlNet is a neural architecture that adds spatial conditioning controls to pretrained text-to-image diffusion models; it can use controls such as edges, depth, segmentation, and pose. [25]

Purpose

Inject spatial structure into diffusion generation.

I/O

```
control image/map + latent + timestep + text conditioning
→ control residuals for denoising model
```

Examples of control inputs

- Canny edges
- depth maps
- normal maps
- human pose skeletons
- segmentation maps
- line art
- scribbles

Mathematical role

ControlNet produces additional residual features that condition the main denoising network.

3.20 IP-Adapter

Type: learned image-prompt adapter
Neural network? yes / adapter module
Common ecosystems: diffusion

Purpose

Use an image as conditioning, similar to a visual prompt.

I/O

```
reference image → image embedding → conditioning for diffusion model
```

Role

It enables image-based style, identity, composition, or concept guidance without fully replacing the text prompt.

3.21 Textual inversion / embeddings

Type: learned token embedding
Neural network? small learned vector, not full model
Common ecosystems: Stable Diffusion

Purpose

Teach a new concept as one or more learned embedding vectors.

I/O

```
special token → learned embedding vector
```

Difference from LoRA

- Textual inversion modifies the prompt embedding space.
- LoRA modifies model weights.
- Fine-tuning modifies many model weights.

3.22 RAG retriever

Type: search / retrieval module
Neural network? often partly, if embeddings or rerankers are neural
Common ecosystems: LangChain, LlamaIndex, OpenAI file search, Azure Foundry, Bedrock, Vertex AI

Purpose

Fetch relevant external knowledge before generation.

I/O

```
query → retrieved chunks
```

Mathematical view

Dense retrieval:

```
q = embed(query)
d_i = embed(document_chunk_i)
score_i = sim(q, d_i)
top_k = arg top_k(score_i)
```

Parameters

- chunk size
- chunk overlap
- embedding model
- vector index
- similarity metric
- top-k
- reranker
- filters / metadata constraints

3.23 Reranker

Type: relevance scoring model

Neural network? often yes

Common ecosystems: RAG

Purpose

Improve retrieval order by scoring query-document pairs more accurately than vector similarity alone.

I/O

```
query + candidate chunks → relevance scores → reordered chunks
```

Mathematical view

```
score_i = fθ(query, chunk_i)
```

3.24 Tool/function caller

Type: structured action interface

Neural network? no, but selected by neural model

Common ecosystems: OpenAI APIs, LangChain, Semantic Kernel, Azure Foundry, Bedrock Agents

Purpose

Allow a model or agent to call external functions.

I/O

```
conversation context + tool definitions
→ JSON tool call
→ external function result
→ model continuation
```

Message type

```
{
  "name": "search_database",
  "arguments": {
    "query": "..."
  }
}
```

Conceptual role

Tool calling turns text generation into controlled service orchestration.

3.25 Agent

Type: high-level orchestration controller

Neural network? mixed; usually LLM + controller logic + tools

Common ecosystems: OpenAI, Azure Foundry, Bedrock, LangChain, Semantic Kernel, SGLang

Purpose

Use model reasoning and tool calls to complete multi-step tasks.

I/O

```
goal + context + tools + memory → actions + observations → final result
```

Components

- instructions
- model endpoint
- tool registry
- memory / state
- planning loop
- retrieval
- evaluator / guardrail
- output schema

---

4. Message and data types

This is the “language” spoken between orchestration elements.

4.1 Text string

```
human-readable symbolic sequence
```

Used by:

- prompts
- chat messages
- documents
- tool descriptions
- instructions

4.2 Token IDs

```
integer sequence
```

Used by:

- LLMs
- text encoders

- tokenizers

Mathematical form:

```
[t_1, t_2, ..., t_n], t_i ∈ ■
```

4.3 Embeddings

```
dense semantic vector
```

Used by:

- RAG
- CLIP
- text encoders
- retrieval
- clustering

Mathematical form:

```
e ∈ ■^d
```

4.4 Hidden states

```
contextual token representations
```

Used inside transformers.

Mathematical form:

```
H ∈ ■^(sequence_length × hidden_dimension)
```

4.5 Logits

```
unnormalized scores over vocabulary/classes
```

Used by LLM samplers and classifiers.

Mathematical form:

```
logits ∈ ■^V
```

where `V` is vocabulary size.

4.6 Probabilities

```
normalized distribution
```

Usually:

```
p = softmax(logits)
```

4.7 KV cache

```
stored attention keys and values from previous tokens
```

Used for efficient autoregressive decoding.

Mathematical form:

```
K,V ∈ ■^(layers × heads × sequence_length × head_dim)
```

4.8 Latent

```
compressed continuous representation of image/video/audio
```

Used by latent diffusion and flow models.

Mathematical form:

```
z ∈ ■^(H/f × W/f × C)
```

4.9 Noise tensor

```
random initial latent state
```

Usually sampled as:

```
z_T ~ N(0, I)
```

4.10 Conditioning vector / conditioning sequence

```
semantic guidance signal
```

Examples:

- CLIP text embedding
- T5 hidden states
- image embedding
- class label embedding
- style embedding

Used by denoisers through cross-attention, adaptive normalization, or concatenation.

4.11 Attention mask

```
which tokens may attend to which tokens
```

Examples:

- causal mask
- padding mask
- cross-attention mask

4.12 Attention map

```
soft routing matrix
```

Mathematical form:

```
A = softmax(QK■ / √d)
```

4.13 Image tensor

```
pixel-space representation
```

Common shapes:

```
H × W × 3
B × C × H × W
```

4.14 Control map

```
structured spatial conditioning input
```

Examples:

- edge map
- depth map
- normal map
- pose skeleton
- segmentation map

4.15 Retrieved chunk

```
text/document fragment selected from external memory
```

Usually contains:

- content
- metadata
- score
- source reference

4.16 Tool call JSON

```
structured action request
```

Example:

```
{
  "tool": "get_weather",
  "arguments": {
    "location": "Brno"
  }
}
```

4.17 Agent state

```
persistent workflow context
```

May include:

- conversation history
- scratchpad
- tool results
- plan
- memory
- active constraints
- pending tasks

4.18 Evaluation trace

```
observed behavior + metrics + judgments
```

Used by platform-level evaluation systems.

May contain:

- prompt
- response
- retrieved context
- tool calls
- latency
- cost
- safety score
- groundedness score
- task completion score

---

5. How the important concepts fit together

5.1 LLM stack

```
User text
  ↓
Tokenizer
  ↓
Token IDs
  ↓
Transformer + KV cache
  ↓
Hidden states
  ↓
LM head
  ↓
Logits
  ↓
Sampler
```

```
  ↓
Next token
  ↓
Detokenizer
  ↓
Text response
```

Typical containers

- GGUF
- safetensors
- PyTorch `.bin`
- ONNX
- TensorRT-LLM artifacts

Typical orchestration systems

- Transformers
- Ollama
- llama.cpp
- vLLM
- SGLang
- TGI
- OpenAI / Azure / Bedrock / Vertex hosted APIs

5.2 Diffusion image stack

```
Prompt text
  ↓
Tokenizer
  ↓
Text encoder, e.g. CLIP/T5
  ↓
Conditioning vectors
  ↓
Initial noise latent
  ↓
Denoiser, e.g. UNet/DiT/FLUX transformer
  ↓
Scheduler / sampler loop
  ↓
Final latent
  ↓
VAE decoder
  ↓
Image
```

Typical containers

- safetensors
- PyTorch checkpoints
- Diffusers repository layout
- ONNX
- TensorRT engines
- GGUF variants for some quantized diffusion components

Typical orchestration systems

- Diffusers
- ComfyUI
- AUTOMATIC1111
- InvokeAI
- Stability Matrix-managed UIs
- cloud image APIs

5.3 RAG / agent stack

```
User query
  ↓
Embedding model
  ↓
Query embedding
  ↓
Vector search / retriever
  ↓
Retrieved chunks
  ↓
Prompt construction
  ↓
LLM
  ↓
Tool calls, optionally
  ↓
Final answer
```

Typical containers

- embedding model weights: safetensors / ONNX / hosted endpoint
- vector database index: FAISS / HNSW / proprietary index
- LLM: GGUF / safetensors / hosted endpoint

Typical orchestration systems

- LangChain
- LlamaIndex

- Haystack
- OpenAI file search / vector stores
- Azure AI Foundry
- Bedrock Knowledge Bases
- Vertex AI

---

6. The “AI Foundry & co.” view

Cloud AI studios and foundries sit above model formats and tensor runtimes.

Their native objects are usually:

```
model endpoint
agent
tool
knowledge base
vector store
prompt
workflow
evaluator
deployment
monitor
trace
dataset
```

not:

```
UNet tensor
VAE latent
GGUF block
CUDA kernel
```

However, they may support importing or deploying models through lower-level formats.

6.1 What platforms usually load directly

Platform type	Usually loads
local PyTorch app	safetensors, `.pt`, `.pth`, `.bin`
ComfyUI	safetensors, ckpt, GGUF via plugins, images/video/audio
Ollama	GGUF / model bundles
llama.cpp	GGUF
ONNX Runtime app	ONNX
TensorRT app	ONNX → TensorRT engine, or prebuilt engine
Azure ML / Vertex / SageMaker	PyTorch, TensorFlow, ONNX, MLflow, custom containers
OpenAI / Anthropic API	usually no raw model import; service endpoints
Azure AI Foundry / Bedrock / Vertex	model catalog + endpoints + some custom model import options

---

6.2 Platform-level orchestration elements

Element	Semantic role
Model endpoint	hosted neural inference service
Prompt	instruction/control message
Agent	high-level controller over model + tools
Tool	external callable capability
Knowledge base	managed retrieval memory
Vector store	embedding-indexed memory
Evaluator	quality/safety measurement module
Guardrail	policy and safety controller
Deployment	production serving object
Trace	execution record
Dataset	examples for tuning/evaluation
Fine-tuning job	parameter adaptation process

6.3 Platform-level message types

Message type	Meaning
chat message	conversational state
system/developer instruction	high-priority control prompt
user message	task request
assistant message	model response
tool definition	schema of callable action
tool call	structured action request
tool result	observation returned to model
file	external document/data artifact
vector store entry	embedded memory object
evaluation result	quality/safety measurement
trace span	recorded execution segment

---

7. Key conceptual corrections

7.1 File format is not runtime

Example:

```
GGUF ≠ llama.cpp ≠ Ollama
```

- GGUF is a file/container format.
- llama.cpp is a runtime/inference engine.
- Ollama is a model manager and serving orchestration layer.

7.2 Diffusers is not PyTorch

```
Diffusers
    ↓
PyTorch
    ↓
CUDA / ROCm
```

- PyTorch executes tensor math.
- Diffusers defines semantic diffusion pipelines.

7.3 VAE, LoRA, scheduler, and FLUX are not the same kind of object

Thing	Correct category
VAE	learned transform module
LoRA	low-rank parameter adapter
Scheduler	numerical controller
FLUX	model family / architecture ecosystem
Latent	message/data type
GGUF	file/container format
ComfyUI	graph orchestration UI/runtime
Azure AI Foundry	application/platform orchestration

7.4 Some modules are neural networks, some are controllers

Neural transform	Algorithmic/controller
VAE	scheduler
UNet	tokenizer
DiT	sampler
CLIP/T5	CFG formula
embedding model	vector index lookup
reranker	tool-call dispatcher

7.5 LLM systems are externally monolithic, diffusion systems are externally modular

LLM stack:

```
tokenizer → transformer → logits → sampler
```

Diffusion stack:

```
text encoder + VAE + denoiser + scheduler + adapters + control modules
```

That is why LLMs often feel more “packaged”, while ComfyUI-style diffusion systems expose many components.

---

8. Compact reference tables

8.1 Major packaging ecosystems

Format / package	Main use	Contains	Typical runtime
safetensors	PyTorch weights	named tensors	PyTorch, Diffusers, Transformers
`.pt` / `.pth`	PyTorch checkpoint	tensors / objects	PyTorch
GGUF	local LLM inference	quantized tensors + metadata/tokenizer	llama.cpp, Ollama
ONNX	portable graph	graph + weights	ONNX Runtime, TensorRT, OpenVINO
TensorRT engine	optimized NVIDIA inference	compiled engine	TensorRT
SavedModel	TensorFlow deployment	graph + variables	TensorFlow Serving
`.keras` / H5	Keras model	config + weights	Keras/TensorFlow
TFLite	mobile inference	flatbuffer graph + weights	TFLite
Core ML	Apple deployment	graph + weights	Core ML

OpenVINO IR	Intel deployment	optimized graph + weights	OpenVINO
MLX	Apple Silicon local AI	arrays + configs	MLX
NCNN/MNN	mobile/edge	optimized graph + weights	NCNN/MNN

8.2 Major semantic orchestration ecosystems

System	Domain	Main abstraction
Diffusers	diffusion	pipeline of components
Transformers	LLM/multimodal	model + tokenizer + generation
ComfyUI	visual generation	node graph
AUTOMATIC1111	image generation	fixed UI pipeline
InvokeAI	creative image workflows	canvas/workflow
Ollama	local LLM	packaged model server
llama.cpp	local LLM runtime	token inference
vLLM	LLM serving	request/KV scheduling
SGLang	structured LLM programs	generation runtime/programming
TGI	LLM serving	API inference server
LangChain	LLM apps	chains/agents/tools
LlamaIndex	RAG/data	indexes/query engines
Haystack	RAG/search	pipelines
DSPy	LLM optimization	declarative modules
Semantic Kernel	enterprise LLM apps	skills/plugins/planners
OpenAI Platform	hosted AI	responses/tools/vector stores
Azure AI Foundry	enterprise AI	agents/models/evals
Bedrock	hosted FM platform	models/agents/KBs
Vertex AI	cloud AI platform	model garden/deployments

8.3 Important transformation modules

Module	I/O	Type
Tokenizer	text $\leftrightarrow$ token IDs	algorithmic
Embedding model	text $\rightarrow$ vector	neural
Transformer	tokens/hidden states $\rightarrow$ hidden states	neural
Attention	Q,K,V $\rightarrow$ context	neural operation

| LM head | hidden → logits | neural |

| Sampler | logits → token | algorithmic |

| VAE encoder | image → latent | neural |

| VAE decoder | latent → image | neural |

| UNet | noisy latent → denoising prediction | neural |

| DiT | latent patches → denoising/flow prediction | neural |

| FLUX model | noise/latent + conditioning → flow | neural model family |

| Scheduler | latent + prediction + t → next latent | numerical |

| CLIP/T5 | text → conditioning | neural |

| LoRA | W → W + low-rank ΔW | adapter |

| ControlNet | control map → control residuals | neural |

| IP-Adapter | image → visual conditioning | neural adapter |

| Retriever | query → chunks | search/neural hybrid |

| Reranker | query+chunks → scores | often neural |

| Tool caller | context → JSON action | orchestration |

8.4 Important message types

| Message type | Used in |

| --- | --- |

| text | prompts, documents, chat |

| token IDs | LLMs, text encoders |

| embeddings | RAG, CLIP, search |

| hidden states | transformers |

| logits | LLM output |

| probabilities | sampling |

| KV cache | LLM serving |

| latents | diffusion |

| noise tensors | diffusion initialization |

| conditioning vectors | diffusion/CLIP/T5 |

| attention masks | transformers |

| attention maps | interpretability/control |

| image tensors | vision/diffusion |

| control maps | ControlNet |

| retrieved chunks | RAG |

| tool call JSON | agents |

| agent state | workflows |

| evaluation traces | platforms |

---

9. Final mental model

A modern AI application is best understood as:

```
model containers
    loaded by
runtimes
    orchestrated by
semantic frameworks
    composed from
transformation modules
    communicating through
typed semantic messages
    exposed as
application services / agents / workflows
```

Or shorter:

```
AI systems are semantic tensor-message pipelines wrapped in service orchestration.
```

References

[1] Hugging Face Safetensors documentation: https://huggingface.co/docs/safetensors/index

[2] GGML GGUF documentation: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md

[3] Hugging Face GGUF documentation: https://huggingface.co/docs/hub/en/gguf

[4] ONNX Concepts documentation: https://onnx.ai/onnx/intro/concepts.html

[5] NVIDIA TensorRT documentation: https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html

[6] NVIDIA TensorRT-LLM GitHub: https://github.com/NVIDIA/TensorRT-LLM

[7] Hugging Face Diffusers `DiffusionPipeline`: https://huggingface.co/docs/diffusers/using-diffusers/loading

[8] Hugging Face Diffusers GitHub: https://github.com/huggingface/diffusers

[9] Hugging Face Transformers documentation: https://huggingface.co/docs/transformers/en/index

[10] Hugging Face Transformers pipelines: https://huggingface.co/docs/transformers/en/main_classes/pipelines

[11] llama.cpp GitHub: https://github.com/ggml-org/llama.cpp

[12] vLLM documentation: https://docs.vllm.ai/en/latest/

[13] SGLang documentation: https://sgl-project.github.io/

[14] OpenAI tools documentation: https://developers.openai.com/api/docs/guides/tools

[15] OpenAI file search documentation: https://developers.openai.com/api/docs/guides/tools-file-search

[16] Microsoft Foundry Agent Service overview: https://learn.microsoft.com/en-us/azure/foundry/agents/overview

[17] Microsoft Foundry observability documentation: https://learn.microsoft.com/en-us/azure/foundry/concepts/observability

[18] Amazon Bedrock Knowledge Bases: https://aws.amazon.com/bedrock/knowledge-bases/

[19] Amazon Bedrock custom model import: https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html

[20] Google Vertex AI documentation: https://docs.cloud.google.com/vertex-ai/docs

[21] Demystifying FLUX architecture: https://arxiv.org/html/2507.09595v1

[22] Scaling Rectified Flow Transformers for High-Resolution Image Synthesis: https://arxiv.org/abs/2403.03206

[23] Hugging Face Diffusers scheduler overview: https://huggingface.co/docs/diffusers/en/api/schedulers/overview

[24] LoRA paper: https://arxiv.org/abs/2106.09685

[25] ControlNet paper: https://arxiv.org/abs/2302.05543